Homework 3 : Symbolic Planning

Gautam Balaji

October 30, 2026

1 Compiling and Running

All of my code is in `code/src/planner.cpp`, below the provided parser. The provided `main()` and `CMakeLists.txt` are unchanged. `code/envs/` contains the four provided environments and my new one, `Warehouse.txt`. From the `code` directory:

```
mkdir build && cd build
cmake .. && cmake --build . --config Release
# or, instead of CMake: g++ -std=c++14 -O2 ../src/planner.cpp -o planner
./planner Blocks.txt 1
```

The arguments provided are the environment to generate the plan for, and the heuristic chosen for the planner: **1** = no heuristic ($h = 0$, the default), **2** = goal count, **3** = empty-delete-list. The planner prints the planning time, the number of expanded states, and the plan, or `No plan found` with an empty plan if none exists.

2 Implementation

The planner is domain-independent and only uses what the parser puts in the `Env` object. The steps of generating the plan can be broken down as follows:

1. Before searching, every action schema is grounded: for a schema with k parameters, all ordered selections of k *distinct* symbols are generated by recursive backtracking, and each one is substituted into the preconditions and effects. Arguments that are not parameters (e.g. `Table`) are treated as constants, and the effects are split into an add list and a delete list.
2. A state is the set of true grounded literals (closed-world assumption), stored in an `unordered_set`, and a state is a goal state if it contains every goal literal.
3. The search is A* with a cost of 1 per action. An action is applicable if all its positive preconditions are in the state and none of its negated ones are, and applying it gives $s' = (s \setminus \text{del}) \cup \text{add}$. Duplicate states are detected with a hash map from state to node, using an order-independent hash (the sum of the literal hashes).
4. The goal test is done when a node is expanded, and the plan is rebuilt from parent indices.

The following heuristics were defined for the planner:

1. **Goal count**: This returns the number of goal literals not in the state, which is not admissible.
2. **Empty-delete-list** ignores all delete effects and repeatedly applies every applicable action to a growing set of facts, and returns the number of rounds (layers) until the goal is reached, or a large constant if no new facts can be added. It is admissible. Since the state space is finite and no state is expanded twice, the search is complete, and with $h = 0$ or empty-delete-list it returns plans with the minimum number of actions. I assume that the parameters of one action always take different symbols (the $b \neq y$ condition from the lecture).

3 Results

The results shown below in Table. (1) were generated after an average of three runs for each planner per environment. Time is measured from the start of the search (after grounding) until the plan is returned. A state counts as expanded when it is removed from OPEN and its successors are generated.

	No heuristic		Goal count		Empty-delete-list		
Environment	Expanded	Time (s)	Expanded	Time (s)	Expanded	Time (s)	Plan length
Blocks	16	0.0050	6	0.0034	5	0.0068	3
BlocksTriangle	702	0.357	75	0.035	115	0.462	6
FireExtinguisher	343	0.144	343	0.141	284	0.320	21
Warehouse (new)	398	0.126	361	0.132	269	0.229	17

Table 1: Number of states expanded and planning time, with and without a heuristic

Here are some inferences that I have drawn from the results shown above:

- All three planners returned plans of the same minimum length in every environment.
- With goal count, the number of expanded states dropped by 63% on Blocks and by 89% on BlocksTriangle, where the search became about 10 times faster. On FireExtinguisher it did not help at all, since there is a single goal literal, `ExtThree(F)`, so $h = 1$ in every state except the goal. The time difference there is run-to-run variation (the same run measured between 0.055 s and 0.141 s).
- Empty-delete-list was the only heuristic that reduced the expansions on every environment (69%, 84%, 17% and 32%), since it estimates how many steps are still needed rather than how many goals are missing. It was also slower than no heuristic everywhere, because every generated state runs a small search over all grounded actions.

4 Extra Credit

New environment (`Warehouse.txt`). A robot must deliver two packages to room R4. The four rooms are connected in a line, R1 – R2 – R3 – R4, the door between R2 and R3 is locked, and the key is in R1. The robot can hold only one object at a time, including the key, so it has to fetch the key, unlock the door, put the key down, and then carry the packages one at a time. The actions are `Move(x,y)` (through an open door), `Unlock(x,y)` (while holding the key), `Pick(o,r)` and `Drop(o,r)`. Unlike the provided domains, unlocking changes which moves are possible for the rest of the plan, and the key and packages compete for the robot's hand. The optimal plan has 17 steps (Table 1).

Empty-delete-list heuristic. Implemented as described in Section 2 and selected with `PLANNER = 3`. Its results are in Table 1.